Compiler repeated work: profile, reduce, verify

Created-at: 2026-09-07 12:48 +02:00

Scope and ETA. Investigate cold source compilation, AST copying, reference resolution and repeated traversals. Approximately 45 minutes: 10 minutes instrumentation, 10 minutes profiles and baseline, 15 minutes implementation and correctness, 10 minutes repeat measurements and report. Generation, parsing, lowering, hashing, prepared compilation and destruction remain distinct measurement scopes. Do not remove validation or change hashes.

Authority and checkout. User requested optimization and investigation of repeated calls. Core fetched successfully at 12:42–12:45 using the existing origin with an SSH HostName override for 192.168.1.228; normal DNS failed. Base is freshly fetched origin/main f305ade; branch perf/compiler-repeated-work reuses the existing checkout. Former branch 2d43e82 has three unmerged diagnostic commits and is preserved. Untracked distributed-verification artifacts are preserved.

Discovery. Extend the existing interning Criterion benchmark and ModuleStore integration tests. ir_bench explicitly excludes parsing/lowering; ir_depth_probe measures depth failure, not compilation. Historical CPU optimization and CBOR reports contain no equivalent compiler profile. Ecosystem reader confirms Core owns compilation and is consumed by Graph, Transformation and Python.

Affected files. This four-format plan; `hymeko_core/benches/interning_bench.rs`; `hymeko_core/tests/mir` candidate production files `hymeko_core/src/module_store/module_store.rs`, `hymeko_core/src/ir/lower.rs` `hymeko_core/src/ir/canonical_hash.rs`, `hymeko_core/src/resolution/resolve.rs`; `reports/2026-09-07` and associated numerical, profile and figure evidence under `reports/compiler-repeated-work/`.

Core protection. User approved at the 2026-09-07 continuation: `APPROVED-CORE-EDIT: compiler-repeated` Instrumentation is allowlisted; candidate production files are lockdown-full. Apply the prepared root-AST/dependency-list borrow patch first, keeping imports, hashing and validation unchanged. Compare exact IR output (only timestamp normalized), then cold compilation and cached lookup. Preserve before/after binaries and PDBs. The user's runtime question also requires quoting the existing prepared Dijkstra results separately: compiler speedups are not traversal-kernel speedups.

Interfaces and invariants. Retain public signatures, cache behavior, resolution ambiguity/error order, import namespace order, IR contents and hash bytes. The legacy Merkle pass writes serialized decl_hash and is not redundant merely because another hash pass exists. No dependency changes. If borrowing ASTs requires an API change, keep the public owned-slice API and share the underlying lowering.

Measurement and tests. Criterion release, minimum five post-warmup iterations; save raw samples, median/IQR/worst. Windows samply CPU profile with symbols before selecting a hot spot. Read existing Graph campaign artifacts first. One large production-scale smoke precedes

repeated large runs. Record process peak working set, host and toolchain. Differential whole-IR checks normalize only metadata timestamps and pin both canonical and declaration hashes; import, constants, aliases, empty inputs and failure cases. Run relevant and workspace tests, fmt, clippy, symbols, duplication and preflight. Benchmark same machine and build settings.

Budgets and risk. Maximum 16 GiB RSS, 180 seconds per measured process; the existing 50,000-node source provides a quick discriminating workload, followed by the graph map512 source (30,032,345 bytes) if available through the existing harness. Target measurable reductions without greater than 10 percent time or memory regression. No claimed speedup without profile attribution. Nested imports and ambiguous reference paths can invalidate naive early return or memoization; tests must preserve them. Cloning may contribute memory rather than dominant CPU cost. Retaining cache entries may exchange speed for unbounded memory, so do not extend cache lifetime speculatively.

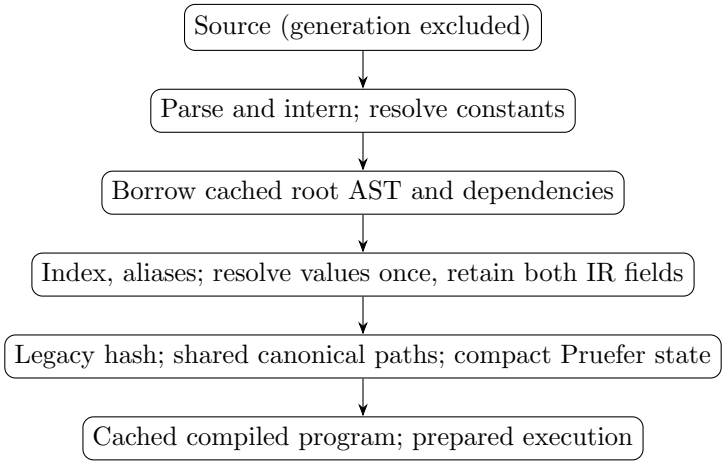
Rollback. Revert only this task's named changes. Preserve user files, previous branches, benchmark data and public hash/wire contracts.

Continuation, 2026-09-07 13:36. Under the same approved slug, strengthen source-to-IR compilation by reusing the canonical path table between validation and hashing. A crate-private CanonicalPaths view borrows the IR and owns just its derived paths; the IR cannot change during its lifetime. Public standalone APIs delegate to the same methods. Validation borrows index paths rather than cloning them, preserving the original index iteration and first-error order. Existing DerivedPathIndex also sorts a lookup table and is therefore inappropriate for this transient compile phase. Extend existing ModuleStore tests to compare compiled and standalone hashes, and to exercise namespace corruption. Frozen hashes and full serialized-IR goldens remain unchanged. Compare the already optimized root-borrow binary against this one change first; only then consider repeated leaf hashing. Expected additional work: 20 minutes including two bounded measurement pairs, tests, coverage and report. Retain the 50k baseline and add a larger existing frontend-generator scale if the smoke fits the declared 16 GiB / 180-second process budget. No second benchmark framework.

Continuation, 2026-09-07 13:59. Profile-guided compact Pruefer state: the resolved final profile puts 19.1 percent of compile-stack samples in prufer_from_forest. Replace its HashMap of dense declaration IDs with a dense vector and per-vertex adjacency lists with degree plus neighbor XOR. For degree one, the XOR equals the sole remaining neighbor, and removing an edge XORs out its endpoint. Preserve the existing heap key/order, labels, token sequence, super-root handling and hash domains. No public API or dependency change; same Core approval. Extend hymeko_core/tests/unified_ir/verification/frozen_hash.rs before production edits with deterministic chains, stars, balanced trees, multiple roots, empty and single-node forests, interleaved anonymous declarations and nondefault hash configuration. Freeze an aggregate digest against the current shared-path version before the rewrite. Reuse its archived binary as the immediate baseline. Measure cold compile and standalone canonical hashing at 50k, then cold compile at the already validated 500k scale in ABBA order; caps remain 180 seconds and 16 GiB per process. Capture a resolved after-profile, execute workspace tests, clippy and coverage, and append to the same report. ETA about 15 minutes: targeted parity/build 3, bounded A/B 6, profile plus checks/report 6. Roll back only this continuation if parity fails or investigation finds a regression. Existing root-borrow and shared-path edits remain.

Continuation, 2026-09-07 20:06. Remove duplicate annotation-value resolution in resolve_signed_refs under the existing `APPROVED-CORE-EDIT: compiler-repeated-work`. Fresh fetch confirms HEAD equals origin/main f305ade; preserve all earlier dirty changes. The exact current profile attributes

610 of 8070 compile samples to this function. Resolve the annotation once and clone its resolved value into the existing weights field; both owned wire fields must remain. Preserve list depth, absent versus empty, floating-point bits, symbol insertion order and first error. Extend existing test_ref_values.rs with mixed-value and error controls before editing production; extend the normalized-IR corpus for the real-parser path. Extend the existing Criterion generator with an explicit mixed-reference payload, retaining the numeric fixture and distinct benchmark identities. Archive matched binaries; compare lowering and cold compile in ABBA order, ten samples after warmup, with raw samples, IQR, worst and process peak. Existing 50k and 500k numeric fixture bounds apply; mixed 50k smoke must precede repeated runs. Each process capped at 180 seconds / 16 GiB. ETA approximately 20 minutes: tests and baseline 5, implementation and measured pairs 7, workspace gates and report/audit 8. No API or dependency changes. Bounded audit covers ModuleStore, intern/constant/reference resolution and lowering; record proven repeated work separately from unmeasured candidates and necessary semantic passes. Other candidates are not automatically bundled into this patch.



PERF-1 integration, 2026-09-09

Created-at: 2026-09-09T18:54:44+02:00. ETA: approximately 60–90 minutes, based on cached builds, existing 1-minute Core tests, 1–3 minute paired benchmark cells, SDK rebuild and browser tests; no complete historical campaign rerun. User approved saving the four-step strategy and executing step 1 in this turn. Existing authorization: `APPROVED-CORE-EDIT: compiler-repeated-work`; the course-module-compilation approval was the user's affirmative reply to that named scope, not a literal token. No new semantic or accelerator work.

Reuse the released course branches (Parser a528d98, Core 8623eb1) as bases. The original dirty performance checkouts remain the preserved source of the measured edits; independent integration worktrees hold the released control and candidate during A/B builds. Core main's independent sparse commit is not silently merged. Retire the temporary worktrees after evidence and publication.

Integrate only the existing grammar/string-scan optimizations, owned-record benchmark adapter, root borrowing/shared canonical paths, compact Pruefer state and single annotation-value resolution, plus their existing tests. Keep final course tests and diagnostics from the published branches. Affected additional paths: Parser grammar/lexer, existing benches/formats, tests/format_comparison.rs, scripts/format_*.py and their tests, Parser Cargo files and existing plan; Core canonical_hash.rs, module_store.rs, resolve.rs, existing interning benchmark and frozen/ref/module-store tests, Cargo pin/lock, generated SYMBOLS.md and docs/hymeko-platform-strategy.md; SDK Cargo pin/lock, existing changelog/package README and generated package. Root report exclusions remain. No Graph/Transformation/Tensor source change.

Use identical harnesses to build released control and integrated candidate. Run large-case

smoke before bounded measurements: 50k linked/long-text parser and 50k mixed Core; 500k Core correctness/resource smoke where affordable. Minimum five post-warmup samples, retain raw values, median/IQR/worst and peak memory, binary/source/fixture hashes, affinity and environment. Reuse existing Campaign/ProcessRunner, Criterion and plotting tools. A/A or repeated control bounds ambient noise. More than ten percent stable regression requires investigation; inconclusive is not pass. Existing profiles justify the already-measured optimization mechanisms, not a fabricated combined speedup. No tests/builds overlap timed cells. Max process peak 16 GiB, timed process 180 seconds. Preserve full-IR and corpus hashes, signed-reference diagnostics, source provenance; run Parser, Core, native/Node/browser/Worker parity and actual packed SDK installation. CPU tests only; no GPU performance claim.

Publication: provider before consumer, immutable Git revisions, one Core identity, no local path patches shipped; verify GitHub and LAN refs. Reuse current course branch without default-branch merge. Archive reports to Samba. Rollback uses previous published SHAs and retained original SDK; never reset or remove unrelated user work. PERF-2/3/4 remain planned at task completion.

Current full-IR characterization, 2026-09-13

Created-at: 2026-09-13 15:37 +02:00. ETA: about 15 minutes for existing target builds, correctness, bounded cells and report. User explicitly requests the full IR benchmark and the compiled library size. No circular-buffer remeasurement, profiler, production change or second compilation pipeline. Reuse ModuleStore::compile through the existing interning benchmark: 50k numeric and mixed sources, cold compile and cache hit reported separately. Reuse existing canonical-CBOR corpus encode/decode cells as independent boundary measurements; do not add phase medians to claim an end-to-end result. Ten or more post-warmup Criterion samples, median/IQR/worst of per-sample iteration averages, no per-call p99 claim. Run the exact public-facade corpus round-trip correctness test first. Use existing measure_peak_memory.ps1 Program mode with explicitly built targets, at most 16 GiB observed working set and 180 seconds per process. A single same-scale smoke precedes repeated cells. Build/check work never overlaps timing. Fresh release library build: default-empty features, no IPC/Arrow/GPU; report rlib versus cdylib/PDB and inspect DLL exports. A small DLL without callable Core exports is not the full linked Rust dependency footprint. Affected tracked files: this existing plan.tex and rebuilt plan.pdf only. Append results to reports/2026-09-07-compiler-repeated-work.md and reuse its compiler-repeated-work evidence directory for raw samples, resource records and plots. Preserve the other session's uncommitted circular-buffer bench, manifest and plan changes. Rollback: revert this plan addendum; no runtime migration.